

CanvasNet Formal Review Report - CanvasNetPdf-PdfDocument

Review that CanvasNetPdf PdfDocument Implementation is Correct

1. Introduction

1.1 Purpose

This document records the formal review of a set of project files.

1.2 Scope

This review covers the PdfDocument software unit of the CanvasNetPdf system — a brand-new, from-scratch PDF tokenizer/parser/content-stream interpreter/renderer delivered across 42 commits on branch `feature/pdf-document-support`. It is the sole direct unit of CanvasNetPdf and has never previously been formally reviewed. The review covers file-structure parsing (tokenizer, object model, xref, page tree), content-stream interpretation (graphics state, path ops), device color, the filter pipeline (LZW/ASCII85/ASCIHex/RunLength/CCITT G4), image and form XObjects, font resolution (Type0/Type1/Type1C/Type3/ToUnicode/fallback), sampled functions, shading and tiling patterns, and text rendering (including this session's new stroke/ fill+stroke render-mode support). Because this unit parses untrusted/adversarial PDF byte streams, the review places particular emphasis on bounds-checking discipline, integer-overflow safety, and whether malformed/adversarial input paths fail closed with a specific typed exception.

Scope caveat (reported separately as a Critical finding, see §3.1): two files that are part of the same PdfDocument unit and implement requirements inside `docs/reqstream/canvas-net-pdf/pdf-document.yaml` — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Encryption.cs` and `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.Type3.cs` (plus their test files `PdfDocumentEncryptionTests.cs` and `PdfDocumentSymbolicEncodingTests.cs`) — are **not** listed in the CanvasNetPdf-PdfDocument review-set's paths: in `.reviewmark.yaml`, confirmed via `dotnet reviewmark --elaborate CanvasNetPdf-PdfDocument` and `dotnet reviewmark --plan --enforce` (which lists both source files as globally uncovered by any review-set). They were reviewed anyway, per

explicit task instruction, to give complete coverage; findings for them are included below but they are **not covered by this review-set's cryptographic fingerprint** and therefore remain formally “uncovered” until `.reviewmark.yaml` is corrected.

1.3 Outcomes

Outcome	Meaning
Pass	The check was performed and the criterion is satisfied
Fail	The check was performed and the criterion is not satisfied
N/A	The check does not apply; justification is required

1.4 Review Details

Field	Value
Project	CanvasNet
Review ID	CanvasNetPdf-PdfDocument
Review Title	Review that CanvasNetPdf PdfDocument Implementation is Correct
Fingerprint	5cf74b0dc492fcbb9e778f2041e3457412e0e011ba4a00a0381bdc73dbfd8140
Review Date	2026-10-02

1.5 Reviewers

Name	Role	Organization	Signature	Date
Formal Review Agent (Copilot CLI)	Automated Formal Reviewer	DEMA Consulting	<i>(automated — no handwritten signature)</i>	2026-10-02

1.6 Files Under Review

File
docs/design/canvas-net-pdf/pdf-document.md
docs/reqstream/canvas-net-pdf/pdf-document.yaml
docs/sysml2/model/canvas-net-pdf/pdf-document.sysml
docs/verification/canvas-net-pdf/pdf-document.md
src/DemaConsulting.CanvasNet.Pdf/NamespaceDoc.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.CcittFax.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Color.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.ContentStream.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Filters.Ascii.cs

File
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Filters.Lzw.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Filters.RunLength.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Filters.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.FontFallback.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.ToUnicode.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.Type0.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.Type1.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Functions.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.GraphicsState.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Images.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.ObjectModel.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.PageTree.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.PathOps.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Patterns.Shading.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Patterns.Tiling.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Patterns.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Text.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Tokenizer.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Xref.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.cs
src/DemaConsulting.CanvasNet.Pdf/PdfPageInfo.cs
test/DemaConsulting.CanvasNet.Pdf.Tests/PdfDocumentTests.cs
test/DemaConsulting.CanvasNet.Pdf.Tests/PdfFixtureTests.cs
test/DemaConsulting.CanvasNet.Pdf.Tests/PdfSystemIntegrationTests.cs
<i>(reviewed additionally, outside configured review-set scope — see §1.2 caveat)</i>
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Encryption.cs
src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.Type3.cs
test/DemaConsulting.CanvasNet.Pdf.Tests/PdfDocumentEncryptionTests.cs
test/DemaConsulting.CanvasNet.Pdf.Tests/ PdfDocumentSymbolicEncodingTests.cs

2. Review Checklist

2.1 Requirements Checks

Applicable: Yes

#	Check	Outcome	Justification
REQ-01	All requirements have a unique identifier	Pass	All 60 requirement entries in pdf-document.yaml use unique CanvasNetPdf-PdfDocument-* IDs (verified by grep; no duplicates).
REQ-02	All requirements are unambiguous	Pass	Each requirement maps to exactly one implementation and one verification subsection with no conflicting interpretation found by any area reviewer.
REQ-03	All requirements are testable	Pass	Every requirement has at least one named, existing, passing test (see §2.13).
REQ-04	All requirements are consistent	Pass	No contradictions found between requirements across areas.
REQ-05	All requirements are complete (no TBDs)	Pass	No TBD/undefined terms found in the requirements file.
REQ-06	All requirements are verifiable	Pass	Verification doc provides a dedicated, objectively-checkable subsection per requirement ID.
REQ-07	No compound requirements are present	Pass	Each requirement expresses a single testable criterion (spot-checked across all seven areas).
REQ-08	No requirements are missing	Fail	Several confirmed, reachable failure modes have no corresponding requirement or test : unbounded xref-stream /w// Index validation (file-structure area), unbounded operand-stack/q-stack growth and non-finite cm/path-coordinate operands (content-stream area), unbounded image/CCITT dimensions and LZW output growth (color/filter area), unbounded composite-font /w CID-range width loop (fonts area), int32 overflow in sampled-function bit-offset arithmetic (patterns/functions area), and encryption /Length bounds validation (encryption area). These are documented below as Critical/High/Medium code findings; the requirements set does not currently obligate a “fails closed

#	Check	Outcome	Justification
			on resource-exhaustion/overflow” behavior for these specific inputs, so there is no regression test to catch them.

2.2 Design Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
DES-01	Design documentation clearly describes the purpose	Pass	pdf-document.md “Purpose” section is clear and matches the unit’s actual role.
DES-02	Design documentation covers necessary implementation details	Pass	Covers data model, key methods, encryption, content-stream interpreter (including color/filter/image/font/text/pattern subsections), and /Pattern color space in detail; all spot-checked subsections matched current source.
DES-03	Design documentation describes how the code is interfaced	Pass	Public API (Open/GetPageInfo/Render/Dispose) documented with inputs/outputs/exceptions.
DES-04	Design documentation describes expected normal operation	Pass	Confirmed across all seven reviewed areas; documented leniency behaviors (shape-sniffing, fallback) match source.
DES-05	Design documentation describes expected error handling	Fail	The “Error Handling” section’s fail-closed guarantees are not fully honored by the implementation in several places — see Critical/High findings in §3.1 (xref /w validation, page-tree/object-model recursion depth, sampled-function overflow, encryption /Length validation) where the actual exception type/behavior diverges from the documented contract.

2.3 Technical Documentation Checks

Applicable: No — this review-set contains no general/user-facing technical documentation (README, user guide, API reference); those are covered by the Purpose and {System}-Architecture review-sets.

2.4 Code Checks

Applicable: Yes

#	Check	Outcome	Justification
CODE-01	Code conforms to project coding standards and style guide	Pass	dotnet build -c Release for the full solution: 0 warnings, 0 errors (verified directly). XmlDoc present on all members sampled across all seven areas.
CODE-02	No obvious resource leaks	Pass	No file-handle/stream/memory-leak findings reported by any area reviewer; Dispose is idempotent and tested.
CODE-03	No hardcoded values that should be configurable	Pass	Constants such as MaxFormNestingDepth, MaxType3NestingDepth (12), MaxFunctionRecursionDepth (32), MaxTileSurfaceDimension (2048) are intentional, documented safety bounds, not configuration smells.
CODE-04	Each unit/function has a single, well-defined responsibility	Pass	File-per-concern partial-class organization (Tokenizer/ObjectModel/Xref/PageTree/ContentStream/GraphicsState/PathOps/Color/Filters/Images/CcittFax/Fonts./FontFallback/Functions/Patterns./Text) cleanly separates responsibilities.
CODE-05	Code is written at an appropriate level of abstraction	Pass	Shared dispatch helpers (PaintCurrentPath, PaintStroke, ResolvePattern) are reused consistently across painting/text/pattern call sites rather than duplicated.
CODE-06	Code has appropriate extensibility	Pass	New font/filter/function types are added via clear switch-based dispatch points; no over-engineering observed.

2.5 Logic Error Checks

Applicable: Yes

#	Check	Outcome	Justification
LOGIC-01	Code does only what is intended	Pass	No unintended side effects found; e.g., q/Q state isolation, cache-before-key encryption ordering, and Type3 nesting-depth isolation are all correctly scoped.
LOGIC-02	All significant inputs and boundary conditions are handled correctly	Fail	Multiple confirmed boundary-condition defects (see §3.1): negative/overflowing xref-stream /w widths causing an unhandled <code>IndexOutOfRangeException</code> ; unbounded page-tree and object-model recursion risking <code>StackOverflowException</code> ; <code>int32</code> multiplication overflow in sampled-function bit-offset arithmetic producing a negative index; unbounded CID-width-range loop; unvalidated encryption /Length producing unhandled exceptions.
LOGIC-03	Concurrency and threading concerns are identified and addressed	N/A	<code>PdfDocument</code> is documented and used as a single-threaded, non-reentrant parse/render pipeline per call; no area reviewer identified any shared mutable static state accessed concurrently other than the documented process-lifetime <code>FallbackFontCache</code> , which is a read-mostly cache outside this review's concurrency-sensitive scope.

2.6 Error Handling & Logging Checks

Applicable: Yes

#	Check	Outcome	Justification
ERR-01	Error handling follows the design documentation	Fail	The documented “fail closed with a specific typed exception” contract is violated by several confirmed paths that instead throw undocumented framework exceptions (<code>IndexOutOfRangeException</code> from malformed xref-stream /w; <code>IndexOutOfRangeException</code> from sampled-function overflow;

#	Check	Outcome	Justification
ERR-02	Logging volume/detail is appropriate	N/A	ArgumentOutOfRangeException/ DivideByZeroException/ CryptographicException from malformed encryption /Length) rather than the class's own InvalidDataException/ UnsupportedImageFeatureException conventions used everywhere else. This unit does not perform logging (no ILogger/tracing dependency); error signaling is exclusively via typed exceptions, which is the documented and appropriate approach for a synchronous parsing/rendering library.
ERR-03	Error messages are user-friendly and actionable	Pass	Sampled exception messages name the offending structural field/feature (e.g., /Length, /O, /U, unsupported feature tokens) rather than generic text.
ERR-04	Error messages/logs do not leak sensitive data	Pass	Verified directly for the encryption area: no password, derived key, or key material appears in any exception message in PdfDocument.Encryption.cs (every throw site inspected).

2.7 Usability / Accessibility Checks

Applicable: Yes (public API ergonomics only — no UI/accessibility surface)

#	Check	Outcome	Justification
USE-01	The feature/API is easy to use correctly	Pass	Open/GetPageInfo/Render/Dispose form a small, conventional IDisposable surface with clear argument validation and ObjectDisposedException guards, confirmed by the file-structure area review.
USE-02	All public APIs are well documented	Pass	XmlDoc present and substantive on all public members sampled.

2.8 Test Checks

Applicable: Yes

#	Check	Outcome	Justification
TEST-01	Tests cover expected (happy-path) behavior	Pass	Every requirement has at least one happy-path test; dotnet test of the Pdf test project: 412/412 pass (net10.0; also confirmed by area reviewers across net8.0/net9.0).
TEST-02	Tests cover error conditions and boundary cases	Fail	Confirmed gaps: no test exists for malformed/negative xref-stream / W or /Index, non-finite cm/path-coordinate operands, pathological operand-stack/q-stack growth, the sampled-function int32 overflow, the unbounded composite-font CID-width-range loop, or the encryption /Length validation gap — all identified as reachable, untested failure modes in §3.1.
TEST-03	Tests are independent and repeatable	Pass	Every area reviewer confirmed tests build their own isolated fixtures with no shared mutable state.
TEST-04	Test names clearly describe the behavior being verified	Pass	Hierarchical {ClassName} _{Method}_{Scenario} _{Expected} naming followed consistently across all four test files, confirmed in every area.
TEST-05	New test cases are added for new functionality/defect fixes	Pass	The newest-landed feature (Tr stroke/fill+stroke render modes) has dedicated, rigorous pixel-level tests (not just exception-absence checks).

2.9 Security Checks

Applicable: Yes

#	Check	Outcome	Justification
SEC-01	No obvious security vulnerabilities	Fail	Multiple confirmed denial-of-service-class vulnerabilities against untrusted PDF input: unbounded image/CCITT / Width×/Height allocation, unbounded LZW output growth (decompression-bomb class), an unhandled DivideByZeroException in the PNG predictor for malformed / Columns//Colors, int32 overflow

#	Check	Outcome	Justification
			in sampled-function bit-offset arithmetic, and an unvalidated encryption /Length field causing unhandled exceptions. None are key-recovery/code-execution vulnerabilities, but they are reachable crash/resource-exhaustion vectors from a malicious/malformed file, which is the primary threat model for a PDF-parsing library. See §3.1/§3.3 for full detail.
SEC-02	Authentication/authorization handled correctly	Pass	Password authentication (user-then-owner dual-attempt, per PDF spec) is implemented correctly for RC4/AESV2/AESV3-R5 per the encryption area's dedicated security review; empty-password and wrong-password paths correctly fail closed.
SEC-03	Sensitive data stored/transmitted securely	Pass	RC4 KSA/PRGA independently verified algorithmically correct; AES/MD5/SHA-256 use real BCL primitives (no hand-rolled crypto besides the spec-mandated RC4, no obsolete APIs, no insecure randomness); no password/key material logged or leaked in exceptions. Minor informational notes (non-constant-time comparison, no explicit key-material zeroing) recorded as Low-severity findings, not blocking.

2.10 Code Readability Checks

Applicable: Yes

#	Check	Outcome	Justification
READ-01	Code is easy to understand	Pass	Consistently clear across all seven areas.
READ-02	Methods/functions are small enough to be understood	Pass	Shared helpers keep painting/dispatch logic compact; no oversized-method findings reported.
READ-03	Symbols are well named	Pass	No naming issues reported.

#	Check	Outcome	Justification
READ-04	Code is located in the correct place in the codebase	Pass	File-per-concern partial-class layout matches the <code>.reviewmark.yaml</code> phase annotations exactly.
READ-05	Flow of control can be easily followed	Pass	Dispatch-table/switch patterns used consistently; no convoluted control flow reported.
READ-06	Data flow is understandable	Pass	No data-flow confusion reported by any area reviewer.
READ-07	Comments provided where code is non-obvious	Fail	<code>PdfDocument.Fonts.cs</code> (<code>StandardGlyphNames</code> field, ~lines 643-646) contains an <code>XmlDoc <remarks></code> comment that self-contradicts the adjacent, correct <code>ApplyDifferences</code> remarks and the actual lenient (no-op, not throw) behavior at <code>Fonts.cs:532-539</code> — a misleading non-obvious-code comment is worse than no comment. See finding in §3.1 (Fonts area).
READ-08	No debug artifacts or commented-out code left in codebase	Pass	None found by any area reviewer.

2.11 Requirements vs Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
REQDOC-01	All reviewed requirements are addressed in the general technical documentation	Pass	The unit design doc addresses every requirement area; no facing doc claims were in scope for this unit review.
REQDOC-02	No reviewed requirement is contradicted by the general technical documentation	Fail	<code>NamespaceDoc.cs</code> (lines ~18-23) and a stale XML-doc render <code>PdfDocument_Render_ValidPageIndex_ReturnsCorrectly</code> test still describe <code>Render</code> as “renders no content”/always be contradicted by the <code>Render/ContentStreamDispatch/every</code> content-stream requirement, all of which document and in content-stream interpretation. The test still passes only be

#	Check	Outcome	Justification
			fixture happens to have no /Contents; the comment, not t wrong.

2.12 Requirements vs Implementation Checks

Applicable: Yes

#	Check	Outcome	Justification
REQIMP-01	All requirements under review are addressed by the implementation	Fail	Within the formally configured file set for this review-set, 7 of the 60 requirements in pdf-document.yaml (EncryptDetection, EncryptionRc4, EncryptionAesV2, EncryptionAesV3R5, EncryptionPasswordAuthentication, Type3FontResolution, Type3GlyphPainting) are not addressed by any file in the paths: list — their sole implementing files (PdfDocument.Encryption.cs, PdfDocument.Fonts.Type3.cs) are absent from the review-set. (Reviewed anyway per task instruction; implementation itself was found credible — see §3.1/§3.5 — but this is a Fail against the strict requirement-to-reviewed-file traceability check, and a configuration defect in .reviewmark.yaml that should be corrected.)
REQIMP-02	No requirement is contradicted by the implementation	Pass	Within the files actually reviewed (including the two out-of-scope files, reviewed anyway), no requirement is contradicted by current source.

2.13 Requirements vs Testing Checks

Applicable: Yes

#	Check	Outcome	Justification
REQTEST-01	Every requirement under review is covered by at least one test	Pass	All 60 requirement IDs trace to at least one named, existing, non-skipped, passing test, confirmed by direct execution (dotnet test: 412/412 for the in-scope project; targeted --filter subsets of 32-120 tests

#	Check	Outcome	Justification
REQTEST-02	Tests verify the behavior described in each requirement	Pass	per area all green) and by reading each named test's body (not just its name) to confirm it exercises the claimed behavior. Sampled tests consistently assert concrete outcomes (rendered pixel colors/positions, specific exception types) rather than vague “no exception thrown” checks — notably strong for the new Tr stroke/fill+stroke tests and the encryption tests (which assert actual decrypted/rendered pixels, not mocks).

2.14 Code vs Design Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
CODEDOC-01	The code correctly implements the design documentation	Pass	Every spot-checked design claim (CTM derivation matrices, dash/line-width device scaling, Type3 nesting guard, Tr render-mode dispatch, shape-sniffing leniencies, encryption algorithm sequencing) matched current source exactly, across all seven areas.
CODEDOC-02	All public APIs/interfaces are documented in the design documentation	Pass	Open/GetPageInfo/Render/Dispose and the /Pattern color-space dispatch are documented with inputs/outputs/exceptions.
CODEDOC-03	Non-obvious algorithms and significant design decisions are explained	Pass	Encryption key-derivation algorithms, shape-sniffing rationale, and recursion-depth guard rationale are all explained in the design doc.
CODEDOC-04	No important code details are missing from the design documentation	Fail	The design doc's “Error Handling” section claims a uniform fail-closed typed-exception contract for malformed input that is not actually upheld for the

#	Check	Outcome	Justification
			specific boundary cases identified in §3.1 (this is a documentation-accuracy issue layered on top of the code defect itself: the doc asserts a guarantee the code does not fully provide).

3. Conclusion

3.1 Summary of Findings

#	Check	Finding
F0	REQIMP-01 (Critical — process/config)	.reviewmark.yaml's CanvasNetPdf-PdfDocument review-set omits src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Encryption.cs, src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.Type3.cs, and their test files (PdfDocumentEncryptionTests.cs, PdfDocumentSymbolicEncodingTests.cs), despite the requirements they implement being inside the reviewed pdf-document.yaml. Confirmed authoritatively via dotnet reviewmark --plan docs/code_review_plan/generated/plan.md --enforce, which reports both source files as globally uncovered by any review-set in the repository (alongside docs/reqstream/quality.yaml, src/DemaConsulting.CanvasNet/Fonts/CffStandardEncoding.cs, temptest/Program.cs, and two CanvasNet.Tests files — a broader repo-wide coverage gap, not unique to this review). Recommendation: add the four missing paths to the CanvasNetPdf-PdfDocument entry in .reviewmark.yaml.
F1	LOGIC-02/ERR-01/SEC-01 (Critical)	PdfDocument.Xref.cs (ParseXrefStreamSection, ~lines 221-284): /w width-triplet values are cast from attacker-controlled PDF numbers with no non-negativity validation. A crafted /w [-1000000 2 1] xref stream drives a negative buffer offset into ReadBigEndian, throwing an unhandled

#	Check	Finding
F2	LOGIC-02/SEC-01 (High)	IndexOutOfRangeException instead of the documented InvalidDataException; the constructor's exception handling does not catch this type, so Open crashes outright rather than falling back to linear scan. No test exists for this input. PdfDocument.PageTree.cs (TraversePageTree, ~lines 61-100): cycle rejection only triggers on a <i>repeated</i> object number; there is no maximum recursion-depth limit (unlike the Form XObject interpreter's documented hard limit of 12). A page tree built from thousands of distinct single-child nodes recurses unbounded and can trigger an uncatchable StackOverflowException. PdfDocument.ObjectModel.cs's mutually recursive array/dictionary parsers have the same unbounded-recursion gap (Medium confidence/severity).
		PdfDocument.Functions.cs (SampledFunction.ReadSample/ReadBits, ~lines 121-153): sampleGroupIndex * outputCount is computed in 32-bit int arithmetic before widening to long. A malformed /FunctionType 0 function with a large (individually legal) /Size and multi-output /Range overflows to a large negative bit offset, bypassing the length guard and throwing IndexOutOfRangeException instead of InvalidDataException. Reproduced by static trace (sampleGroupIndex=600000000, outputCount=4 → negative bitOffset). No test covers this.
F3	LOGIC-02/SEC-01 (High)	Color/filter/image area (per dedicated sub-review): unbounded /Width×/Height (and CCITT /Columns//Rows) allocation enabling memory-exhaustion DoS; an unhandled DivideByZeroException in the PNG predictor for a malformed /Columns//Colors combination; unbounded LZW output growth (decompression-bomb class); unchecked double→int casts from PDF numbers; int overflow in /Hival-based palette-length checks; a raw IndexOutOfRangeException leaking from image pixel decoding on truncated streams. All violate the documented "fail closed with a typed exception" contract for this area.
F4	LOGIC-02/SEC-01 (High, memory/DoS)	

#	Check	Finding
F5	LOGIC-02/SEC-01 (Medium)	<code>PdfDocument.Fonts.Type0.cs</code> (ResolveCompositeWidths, ~lines 391-404): the <code>/W</code> array's <code>cFirst cLast w</code> range form has no upper bound on <code>cLast - firstCid</code>; a malformed <code>/W [0 2147483647 500]</code> entry drives an unbounded dictionary-population loop — a resource-exhaustion vector, not memory-unsafe, but untested.
F6	ERR-01/SEC-01 (Medium)	<code>PdfDocument.Encryption.cs</code> (~lines 259-260, 544, 563, 675): the encryption dictionary's <code>/Length</code> field is used to derive <code>keyLengthBytes</code> with no range validation before slicing a 16-byte MD5 digest; malformed <code>/Length</code> values throw an unhandled <code>ArgumentOutOfRangeException</code> or <code>DivideByZeroException</code> instead of the file's otherwise-consistent <code>InvalidDataException</code> convention. A related, lower-severity issue: <code>Aes.Key</code> is assigned outside the <code>try/catch(CryptographicException)</code> block in <code>DecryptAesCbc</code> (~line 978), so an invalid key size throws an uncaught <code>CryptographicException</code>.
F7	LOGIC-01/Medium (content-stream area)	<code>PdfDocument.GraphicsState.cs/</code> <code>PdfDocument.PathOps.cs</code>: numeric operands for <code>cm</code>, <code>w</code>, <code>M</code>, and path-construction operators are not checked for finiteness (<code>NaN/Infinity</code>); a crafted huge-digit-run token can produce <code>double.PositiveInfinity</code>, poisoning the CTM or <code>MiterLimit</code> silently rather than failing closed, inconsistent with the finite-check already applied to dash-array entries elsewhere in the same file. Unbounded content-stream operand-stack and <code>q</code>-stack growth (no cap) are related, lower-severity findings in the same area.
F8	READ-07/ REQDOC-02 (Low-Medium, documentation-only)	Two stale/self-contradicting doc comments found: (a) <code>NamespaceDoc.cs</code> and a test's <code>XmlDoc</code> still describe <code>Render</code> as rendering no content (“Phase 1” blank-only behavior), contradicted by the fully-implemented content-stream interpreter; (b) <code>PdfDocument.Fonts.cs</code>'s <code>StandardGlyphNames</code> remarks claim unrecognized <code>/Differences</code> glyph names throw <code>InvalidDataException</code>, contradicting the actual (and correctly designed) lenient no-op behavior in

#	Check	Finding
F9	SEC-03 (Low, informational)	ApplyDifferences and that same method's own correct remarks. Neither is a behavioral defect — both are misleading comments that should be corrected for maintainability. Encryption area: password/MAC comparisons use non-constant-time Span.SequenceEqual (low real-world risk for a local-file reader with no timing oracle); derived key material is not explicitly zeroed after use (standard managed-crypto limitation). Recorded for completeness, not blocking.

3.2 Overall Outcome

Overall Outcome: Fail

This review finds the PdfDocument unit to be an impressively thorough, well-tested, and largely spec-correct implementation — requirements traceability is excellent (all 60 requirements map to credible implementations and genuinely-assertive, passing, non-disabled tests), the cryptographic algorithms are independently verified correct, and the newest-landed feature (Tr stroke/fill+stroke render modes) is rigorously tested at the pixel level. However, the overall outcome is **Fail** because:

1. A **Critical** review-set configuration gap (F0) means two in-unit source files implementing seven requirements (encryption and Type3 fonts) are not formally covered by this or any review-set — a .reviewmark.yaml fix is required before this unit's coverage claim is accurate.
2. Multiple **Critical/High** confirmed boundary-condition and integer-overflow defects (F1-F6) allow malformed/adversarial PDF input to crash the process with an undocumented, untyped exception (or exhaust memory/stack) rather than failing closed with the class's own documented InvalidDataException/UnsupportedImageFeatureException conventions — a direct violation of this project's own stated untrusted-input-parser standard, and of the design documentation's own "Error Handling" guarantees. None of these are exploitable for code execution or key recovery, but all are reachable denial-of-service-class defects against a library whose entire purpose is processing untrusted files.

Conditions required before this review can be considered approved:

- Correct the .reviewmark.yaml CanvasNetPdf-PdfDocument path list to include PdfDocument.Encryption.cs, PdfDocument.Fonts.Type3.cs, and their test files (F0).
- Add explicit non-negativity/overflow/upper-bound validation and ensure a typed InvalidDataException is thrown for: xref-stream /w/Index fields (F1), sampled-function bit-offset arithmetic (F3), image/CCITT dimensions and LZW output growth and the PNG predictor divide (F4), composite-font /w CID-range width loops (F5), and the encryption /Length field (F6). Add a recursion-depth guard to page-tree traversal and object-model array/dictionary parsing consistent

with the existing `MaxFormNestingDepth/MaxType3NestingDepth` pattern (F2).
Add regression tests for each.

- Correct the two misleading/stale documentation comments (F8).

No source code or documentation was modified as part of this review; all findings are reported for the orchestrator/maintainers to triage and remediate.